école Centrale

TP 1 - traitement d'images - partie 2

Objectifs du TP

Cette deuxième partie du TP porte sur le filtrage spatial, la morphologie mathématique et l'analyse fréquentielle des images.
À l'issue de cette séance, vous devrez être capables de :

- appliquer un filtre de convolution et interpréter l'effet de son noyau sur une image ;
- comparer le comportement de différents filtres en présence de bruit et choisir un traitement adapté au type de bruit observé ;
- programmer les opérations élémentaires de morphologie mathématique sur une image binaire ;
- interpréter le spectre de Fourier d'une image et réaliser un filtrage fréquentiel simple ;
- confronter des prévisions aux résultats expérimentaux et expliquer les différences observées.

Les programmes réalisés devront être accompagnés d'une analyse des résultats. Il ne s'agit pas uniquement d'obtenir une image transformée, mais de comprendre le rôle des paramètres, des noyaux et des méthodes utilisés.

Rendus attendus

Les modalités de rendu et les règles concernant l'utilisation d'une IA générative sont identiques à celles de la première partie du TP.
Vous rendrez :

- les programmes Python réalisés pendant le TP ;
- un compte rendu au format **PDF** ou **md** présentant les résultats demandés et leur interprétation.

Le compte rendu devra contenir des figures comparatives lisibles, accompagnées de titres, de légendes et des paramètres utilisés. Une succession de captures d'écran sans organisation ni interprétation ne constitue pas un compte rendu.
Vous pouvez utiliser une IA générative pour rechercher une fonction, expliquer une erreur ou proposer du code. Vous restez toutefois responsables du code rendu et devez être capables d'en expliquer le fonctionnement et les résultats.
Dans votre compte rendu, indiquez brièvement les usages significatifs d'une IA générative et présentez au moins un élément de sa réponse que vous avez vérifié, corrigé ou amélioré.

1 Convolution

1.1 Application de filtres de convolution

Une convolution consiste à calculer la valeur de chaque pixel de sortie à partir de son voisinage et d'un noyau de convolution.
Chargez l'image `peppers-512.png` en niveaux de gris, puis appliquez les six noyaux proposés ci-dessous avec la fonction `cv2.filter2D` :

https://docs.opencv.org/4.x/d4/d13/tutorial_py_filtering.html

Vous écrirez une fonction permettant d'appliquer un noyau quelconque afin de ne pas répéter le même code pour chaque filtre.
Avant d'exécuter le programme, examinez les coefficients des noyaux et classez-les, lorsque cela est possible, dans les catégories suivantes :

- lissage ;
- renforcement des détails ;
- détection de contours.

Vous vérifierez ensuite vos prévisions à partir des images obtenues.

$$K_1 = \frac{1}{16}\begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix} \qquad K_2 = \frac{1}{9}\begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

$$K_3 = \begin{pmatrix} 1 & -3 & 1 \\ -3 & 9 & -3 \\ 1 & -3 & 1 \end{pmatrix} \qquad K_4 = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$$

$$K_5 = \begin{pmatrix} 0 & -1 & -1 \\ 1 & 0 & -1 \\ 1 & 1 & 0 \end{pmatrix} \qquad K_6 = \begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

Appliquez successivement les six noyaux à l'image `peppers-512.png`.

Pour les filtres produisant des valeurs négatives, effectuez les calculs dans un type signé ou flottant. Choisissez ensuite une représentation permettant de visualiser le résultat.

Compte rendu

Présentez l'image originale et les six images filtrées dans une ou plusieurs figures comparatives. Indiquez clairement le noyau utilisé pour chaque résultat.
Présentez également, sous la forme d'un tableau, les informations suivantes :

Noyau	Somme des coefficients	Effet prévu	Effet observé
K_1			
K_2			
K_3			
K_4			
K_5			
K_6			

Complétez cette présentation en répondant aux questions suivantes :

1. Quel lien observez-vous entre la somme des coefficients d'un noyau et son effet dans les régions où l'intensité est constante ?
2. Les noyaux K_1 et K_2 réalisent tous les deux un lissage. Quelle différence observez-vous entre leurs résultats ? Expliquez cette différence à partir de la répartition de leurs coefficients.
3. Quel noyau renforce les détails sans supprimer complètement l'image originale ? Justifiez votre réponse à partir de ses coefficients et du résultat obtenu.
4. Parmi les noyaux proposés, lesquels détectent principalement des contours ? Comparez les orientations des contours mises en évidence.
5. Quelle méthode avez-vous utilisée pour représenter les résultats comportant des valeurs négatives ?
6. Pourquoi un résultat codé directement avec des entiers non signés sur 8 bits peut-il être incorrect ou incomplet ?

1.2 Filtrage d'images bruitées

Dans cette partie, vous étudierez deux types de bruit :
- un bruit gaussien, obtenu avec la fonction `add_gaussian_noise` ;
- un bruit impulsionnel, appelé bruit « poivre et sel », obtenu avec la fonction `add_salt_pepper_noise`.

Appliquez à l'image `peppers-512.png` en niveaux de gris :
- un bruit gaussien avec $\mu = 0$ et $\sigma = 25$;
- un bruit poivre et sel avec une proportion de pixels bruités égale à $0{,}05$.

Les deux fonctions suivantes permettent de produire les images bruitées utilisées dans cette partie.

— La fonction `numpy.random.normal` permet de générer un bruit gaussien.

```
def add_gaussian_noise(image, mean=0, std=25):
    noise = np.random.normal(mean, std, image.shape)
    noisy = image.astype(np.float32) + noise
    noisy = np.clip(noisy, 0, 255)
    return noisy.astype(np.uint8)
```

— La fonction `numpy.random.rand` permet de sélectionner aléatoirement les pixels remplacés par du noir ou du blanc dans un bruit « poivre et sel ».

```
def add_salt_pepper_noise(image, amount=0.05):
    noisy = image.copy()
    random_values = np.random.rand(*image.shape)

    noisy[random_values < amount / 2] = 0
    noisy[random_values > 1 - amount / 2] = 255

    return noisy
```

Afin de pouvoir reproduire les mêmes résultats, initialisez le générateur aléatoire avant d'appeler ces fonctions :

```
np.random.seed(0)
```

Filtrez ensuite chacune des deux images bruitées avec :
— le noyau de lissage K_1 étudié précédemment ;
— un filtre médian de taille 3×3, à l'aide de la fonction `cv2.medianBlur`.
Documentation :

https://docs.opencv.org/4.x/d4/d13/tutorial_py_filtering.html

Compte rendu

Présentez deux figures comparatives :
— image originale, image avec bruit gaussien, puis résultats des deux filtrages ;
— image originale, image avec bruit poivre et sel, puis résultats des deux filtrages.
Indiquez dans les légendes les paramètres du bruit et la taille des filtres utilisés.
Interprétez les résultats en répondant aux questions suivantes :

1. Comment les deux types de bruit se manifestent-ils visuellement ? Quelle différence fondamentale observez-vous entre leurs répartitions dans l'image ?
2. Pourquoi l'image est-elle convertie en `float32` avant l'ajout du bruit gaussien, puis limitée à l'intervalle $[0, 255]$ avant d'être reconvertie en `uint8` ?
3. Quel est l'effet du noyau K_1 sur chacun des deux types de bruit ? Quel effet produit-il également sur les contours et les détails de l'image ?
4. Pour quel type de bruit le filtre médian donne-t-il le meilleur résultat ? Expliquez pourquoi à partir du principe de calcul de la médiane dans un voisinage.

5. Pourquoi le filtre médian n'est-il pas un filtre de convolution linéaire ?
6. Pour chaque type de bruit, quel filtre choisiriez-vous ? Justifiez votre réponse en tenant compte à la fois de la réduction du bruit et de la conservation des détails.

2 Morphologie Mathématique

Attention : l'objectif principal de cette partie est la manipulation d'images pixel par pixel. Il ne s'agit donc en aucun cas d'utiliser les fonctions d'OpenCV réalisant directement les mêmes opérations. Vous devrez parcourir les pixels de l'image à l'aide de boucles, comme lors du sous-échantillonnage réalisé dans la première partie du TP.
La morphologie mathématique permet de transformer la forme des objets présents dans une image binaire à l'aide d'un élément structurant.
Dans cette partie, les objets seront représentés par des pixels blancs (valeur 255) et le fond par des pixels noirs (valeur 0).
Chargez l'image `peppers-512.png` en niveaux de gris, puis transformez-la en image binaire par seuillage :

```
_, binary = cv2.threshold(
    image, 128, 255, cv2.THRESH_BINARY
)
```

Vous utiliserez l'élément structurant carré suivant :

$$B = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}.$$

2.1 Érosion et dilatation

Programmez les opérations d'érosion et de dilatation en parcourant l'image pixel par pixel.
Pour chaque position de l'élément structurant :

- le pixel résultant de l'érosion est blanc si tous les pixels du voisinage sélectionné sont blancs ;
- le pixel résultant de la dilatation est blanc si au moins un pixel du voisinage sélectionné est blanc.

Vous ne devez pas utiliser les fonctions `cv2.erode`, `cv2.dilate` ou `cv2.morphologyEx` pour réaliser ces opérations.
Pour les pixels situés à l'extérieur de l'image, vous considérerez que leur valeur est nulle. Indiquez clairement ce choix dans votre programme.

2.2 Ouverture et fermeture

À partir de vos fonctions d'érosion et de dilatation, réalisez :

- une ouverture, composée d'une érosion suivie d'une dilatation ;

— une fermeture, composée d'une dilatation suivie d'une érosion.

Calculez également la différence entre l'image dilatée et l'image érodée. Cette différence constitue un gradient morphologique.

2.3 Gradient morphologique

Le gradient morphologique est défini comme la différence entre la dilatation et l'érosion d'une image :

$$G_B(I) = (I \oplus B) - (I \ominus B),$$

où I désigne l'image binaire, B l'élément structurant, $\oplus$ l'opérateur de dilatation et $\ominus$ l'opérateur d'érosion.

Il met en évidence les pixels situés au voisinage des frontières des objets. Calculez ce gradient à partir des résultats produits par vos propres fonctions d'érosion et de dilatation.

2.4 Vérification

Une fois vos fonctions terminées, comparez leurs résultats à ceux des fonctions correspondantes d'OpenCV. Le programme fourni sur la plateforme pédagogique peut être utilisé pour cette vérification.

Vous pouvez notamment compter le nombre de pixels différents entre deux résultats :

```
difference = np.count_nonzero(
    manual_result != opencv_result
)
print("Nombre de pixels différents :", difference)
```

L'utilisation des fonctions de morphologie d'OpenCV est autorisée uniquement pour cette étape de vérification.

Compte rendu

Présentez dans une figure comparative :

— l'image binaire initiale ;
— l'image érodée ;
— l'image dilatée ;
— l'ouverture ;
— la fermeture ;
— le gradient morphologique.

Indiquez la valeur du seuil, l'élément structurant et la convention utilisée pour traiter les bords.

Répondez ensuite aux questions suivantes :

1. Comment l'érosion et la dilatation modifient-elles la taille des régions blanches ? Qu'arrive-t-il aux petits objets et aux petits trous ?

2. Pourquoi l'ordre des opérations est-il important dans l'ouverture et la fermeture ?
3. Quelles structures sont principalement supprimées par l'ouverture ? Quelles structures sont principalement corrigées par la fermeture ?
4. Quelle information est mise en évidence par la différence entre l'image dilatée et l'image érodée ?
5. Vos résultats sont-ils identiques à ceux d'OpenCV ? S'il existe des différences, où sont-elles situées et comment pouvez-vous les expliquer ?

Bonus

Recommencez une partie des expériences avec l'élément structurant en croix suivant :

$$B_{\text{croix}} = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{pmatrix}.$$

Comparez son effet à celui de l'élément structurant carré.

3 Transformée de Fourier

La transformée de Fourier permet de représenter une image dans le domaine fréquentiel. Les basses fréquences correspondent aux variations lentes de l'intensité, tandis que les hautes fréquences correspondent aux variations rapides, comme les contours, les textures fines ou le bruit.
La transformée de Fourier $F(u, v)$ contient des valeurs complexes. Son spectre d'amplitude est défini par :

$$A(u, v) = |F(u, v)|.$$

Les valeurs de ce spectre peuvent présenter de très grands écarts. Pour faciliter sa visualisation, le programme fourni utilise une représentation logarithmique :

$$A_{\log}(u, v) = \log(1 + |F(u, v)|).$$

L'ajout de 1 permet notamment d'éviter le calcul de $\log(0)$. Cette représentation logarithmique est utilisée uniquement pour l'affichage.
Pour davantage d'explications et d'exemples, vous pouvez consulter les tutoriels officiels d'OpenCV :

— calcul du spectre, filtrage fréquentiel et transformée inverse :
https://docs.opencv.org/4.x/de/dbc/tutorial_py_fourier_transform.html
— présentation détaillée de la transformée de Fourier discrète :
https://docs.opencv.org/4.x/d8/d01/tutorial_discrete_fourier_transform.html

3.1 Programme fourni

Le programme `discrete_fourier_transform.py` contient une fonction `calculateDFTMag` qui :

- complète éventuellement l'image afin d'obtenir des dimensions adaptées au calcul de la DFT ;
- construit une représentation complexe à deux canaux ;
- calcule la transformée de Fourier avec `cv.dft` ;
- calcule le spectre d'amplitude ;
- applique une échelle logarithmique ;
- place les basses fréquences au centre du spectre ;
- normalise le résultat pour permettre son affichage.

Commencez par exécuter ce programme et identifiez dans la fonction les instructions correspondant à chacune de ces étapes.

3.2 Images périodiques

Utilisez la fonction `calculateDFTMag` pour calculer les spectres d'amplitude des images `D1r.pgm`, `D11r.pgm` et `D46r.pgm`.
Pour chaque image, affichez côte à côte l'image originale et son spectre. Utilisez la même méthode d'affichage pour les trois expériences.

3.3 Effet d'une rotation

Chargez l'image `peppers-512.png` en niveaux de gris et calculez son spectre d'amplitude. Effectuez ensuite une rotation de l'image de 45° avec les fonctions `cv.getRotationMatrix2D` et `cv.warpAffine`. Conservez dans un premier temps les dimensions de l'image originale. Calculez le spectre de l'image tournée et comparez-le au spectre de l'image originale.

Bonus

Effectuez une seconde rotation en agrandissant le canevas afin de conserver l'ensemble de l'image tournée. Comparez le résultat et son spectre avec ceux obtenus lorsque les dimensions de l'image restent inchangées.

3.4 Filtrage fréquentiel

La fonction `calculateDFTMag` retourne deux résultats :

- `complexI`, qui contient la transformée de Fourier complexe de l'image ;
- `magI`, qui contient une représentation logarithmique, centrée et normalisée du module, destinée uniquement à l'affichage.

La variable `magI` permet de visualiser le spectre et peut aider à repérer les fréquences à conserver ou à supprimer.
En revanche, le filtrage numérique et la transformée inverse doivent être effectués à partir de `complexI`. Cette variable permet de retrouver le module original et la phase de la transformée.

Le masque sera appliqué au module original, puis l'image sera reconstruite en conservant la phase.

3.4.1 Calcul du module et de la phase

Commencez par placer les basses fréquences au centre de la transformée complexe :

```
complexI_shift = np.fft.fftshift(
    complexI,
    axes=(0, 1)
)
```

Séparez les parties réelle et imaginaire, puis calculez le module et la phase de la transformée :

```
real, imaginary = cv.split(complexI_shift)

magnitude, phase = cv.cartToPolar(
    real,
    imaginary
)
```

La variable `magnitude` contient le module original de la transformée. Elle ne doit pas être confondue avec `magI`, qui a été transformée pour permettre son affichage.

3.4.2 Construction des masques

Un filtre passe-bas conserve les basses fréquences situées au centre du spectre. Un filtre passe-haut conserve les fréquences situées à l'extérieur de cette région centrale.
Construisez un masque passe-bas conservant un carré central de demi-largeur 30 pixels :

```
rows, cols = magnitude.shape
cy, cx = rows // 2, cols // 2
radius = 30

low_pass_mask = np.zeros(
    (rows, cols),
    dtype=np.float32
)

low_pass_mask[
    cy-radius:cy+radius,
    cx-radius:cx+radius
] = 1
```

Construisez ensuite le masque passe-haut complémentaire :

```
high_pass_mask = 1 - low_pass_mask
```

Appliquez les deux masques au module de la transformée :

```
low_pass_magnitude = magnitude * low_pass_mask
high_pass_magnitude = magnitude * high_pass_mask
```

La phase de la transformée originale est conservée dans les deux cas.

3.4.3 Reconstruction des images

Utilisez la fonction suivante pour reconstruire une image à partir d'un module filtré et de la phase originale :

```
def reconstruct_image(filtered_magnitude, phase, image_shape):
    real, imaginary = cv.polarToCart(
        filtered_magnitude,
        phase
    )

    filtered_complex = cv.merge([
        real,
        imaginary
    ])

    filtered_complex = np.fft.ifftshift(
        filtered_complex,
        axes=(0, 1)
    )

    reconstructed = cv.idft(
        filtered_complex,
        flags=cv.DFT_SCALE | cv.DFT_REAL_OUTPUT
    )

    rows, cols = image_shape
    return reconstructed[:rows, :cols]
```

Utilisez cette fonction pour reconstruire les résultats des deux filtrages :

```
low_pass_image = reconstruct_image(
    low_pass_magnitude,
    phase,
    I.shape
)

high_pass_image = reconstruct_image(
    high_pass_magnitude,
    phase,
    I.shape
)
```

L'image obtenue après le filtrage passe-haut peut contenir des valeurs positives et négatives. Pour la visualiser, affichez la valeur absolue normalisée :

```
high_pass_display = cv.normalize(
    np.abs(high_pass_image),
    None,
    0,
    1,
    cv.NORM_MINMAX
)
```

Compte rendu

Présentez dans des figures comparatives :

- les images D1r.pgm, D11r.pgm et D46r.pgm, accompagnées de leurs spectres ;
- l'image peppers-512.png, l'image tournée et leurs spectres respectifs ;
- les masques passe-bas et passe-haut ;
- les images reconstruites après les deux filtrages.

Indiquez dans les légendes l'angle de rotation et la dimension de la région centrale utilisée pour construire les masques.
Interprétez les résultats en répondant aux questions suivantes :

1. Où se trouvent les basses et les hautes fréquences dans les spectres affichés ? Que représente le point situé au centre ?
2. Quel lien observez-vous entre la périodicité d'un motif et la position des points correspondants dans son spectre ?
3. Quel lien existe entre l'orientation des structures d'une image et celle des structures visibles dans son spectre ?
4. Comment la rotation de l'image se traduit-elle dans le domaine fréquentiel ? Quels effets supplémentaires sont produits par le découpage et les bordures de l'image tournée ?
5. Quel est l'effet du filtre passe-bas sur les détails et les contours ? Quel est l'effet du filtre passe-haut ? Observez-vous des artefacts produits par la transition brutale des masques carrés ?
6. Pourquoi le masque est-il appliqué au module original et non à sa représentation logarithmique utilisée pour l'affichage ?
7. Pourquoi faut-il conserver la phase lors de la reconstruction de l'image ?

Bonus : suppression de structures orientées

Choisissez une image contenant des lignes horizontales ou verticales. Repérez les structures correspondantes dans son spectre, puis construisez un masque permettant de les atténuer.
Présentez l'image initiale, le masque fréquentiel et l'image reconstruite. Expliquez le lien entre l'orientation des lignes dans l'image et la position des fréquences masquées.